# Iterator

Iterator · Round-robin assign jobs to workers · Build with Waqar

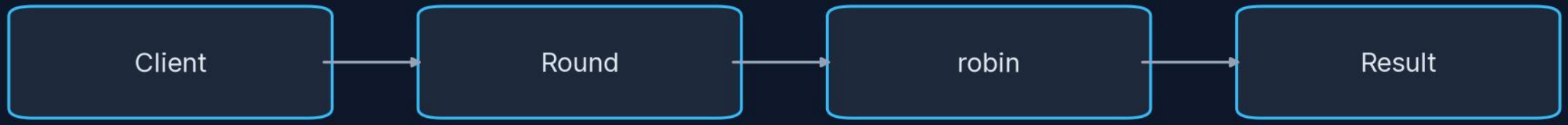
# Today's Hook

- Problem: Round-robin assign jobs to workers
- Pattern: Iterator — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

# The Problem

- Round-robin assign jobs to workers
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

# Structure




Iterator: Round-robin assign jobs to workers

# Real-World Scenario

- Running example: Round-robin assign jobs to workers
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

# Naive Code

```java
package com.buildwithwaqar.patterns.gof.behavioral.iterator;

import com.buildwithwaqar.patterns.gof.behavioral.iterator.RoundRobin;
import java.util.List;
/** Problem: distribute jobs fairly across workers forever. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Iterator problem: round-robin workers ===");
    var it = new RoundRobin<>(List.of("w1","w2","w3")).iterator();
    for (int i=0;i<5;i++) System.out.println("job"+i+"->" + it.next());
  }
}
```

# Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

# Solution

```java
package com.buildwithwaqar.patterns.gof.behavioral.iterator;

import java.util.Iterator;
import java.util.List;
import java.util.NoSuchElementException;

/** Custom iterator that cycles through a non-empty list. */
public final class RoundRobin<T> implements Iterable<T> {
    private final List<T> items;

    public RoundRobin(List<T> items) {
        if (items.isEmpty()) throw new IllegalArgumentException("empty");
        this.items = List.copyOf(items);
    }

    @Override
    public Iterator<T> iterator() {
        return new Iterator<>() {
            int i = 0;
            public boolean hasNext() { return true; }
            public T next() {
                if (items.isEmpty()) throw new NoSuchElementException();
                var v = items.get(i % items.size());
                i++;
                return v;
            }
```

# Solution (2)

```
        };
    }
}
```

# Runnable main

```java
package com.buildwithwaqar.patterns.gof.behavioral.iterator;

import com.buildwithwaqar.patterns.gof.behavioral.iterator.RoundRobin;
import java.util.List;
/** Problem: distribute jobs fairly across workers forever. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Iterator problem: round-robin workers ===");
    var it = new RoundRobin<>(List.of("w1","w2","w3")).iterator();
    for (int i=0;i<5;i++) System.out.println("job"+i+"->" + it.next());
  }
}
```

# Walkthrough Output

```
=== Iterator problem: round-robin workers ===
job0->w1
job1->w2
job2->w3
job3->w1
job4->w2
```

# Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

# Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

# Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Iterator.

# Related Patterns

- Contrast Iterator with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

# Recap

- Problem → naive → Iterator → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.